



CREATING AN ARTIFICIAL NEURAL NETWORK (CNN) ON MATLAB FOR CLASSIFICATION OF RICE IMAGES ACCORDING TO VARIETY

PRATHAM PRAKASH

Methodology Snapshot

What is our way forward?



Introduction - understanding the problem

Our aim

We intend to develop an Artificial Neural Network which is capable of accurately classifying images of rice grains from various varieties into their respective groups or labels.

What do we understand so far?

A simple classification problem - assign input data into predefined categories or classes available.

Solution - way forward

- We will create an Artificial Neural network (CNN) and train it to segregate images of rice grains into their respective variety.
- The training of the model will be done based on the provided data set of rice grain images.
- We will not be creating a model from scratch but rather build a secondary model on top of a primary pre-trained model available on MATLAB.

Understanding the Data

Preliminary analysis

Dataset size

We have a total of 75,000 images (labelled dataset) - 15,000 per class available for training and testing our model



Ipsala



Arborio



Karacadag



Jasmine



Basmati

Resolution, format, and size

Arborio - JPG - 3.5 Kbs - 250 * 250

Basmati - JPG - 3.5 Kbs - 250 * 250

Ipsala - JPG - 4.5 Kbs - 250 * 250

Jasmine - JPG - 3.5 Kbs - 250 * 250

Karacadag - JPG - 3.4 Kbs - 250 * 250

Key note

For ease of experimentation, we will **reduce the data of 75,000 to 500 images - 100 per class.**

This ensures a much faster convergence of models (**from 3.2 hours to 30 mins approximately**)

This is required while working with CNNs as we need to test out different learning parameters to figure out the best working synergy of learning parameters

This allows us to draw **conclusions in an efficient, informed and most of all overcome the problem of shortage of time. We can then test them with the entire 75,000 dataset**

Assessing the Performance of our models - CNNs

We have mainly focused on 3 umbrella performance metrics - Accuracy, Time taken, and Memory

Under Accuracy we have

Macro Accuracy

The average accuracy across all rice types. It provides an overall view of the model's performance.

Macro Precision

Average precision across all rice types. It shows how often the model's predictions are correct.

Macro Recall

Average recall across all rice types. It shows how well the model detects all rice types.

Macro F1 Score

A balance between precision and recall across all rice types. It indicates the model's ability to be both precise and complete.

Overall Validation Accuracy

The percentage of correct predictions on new images. It measures the model's ability to correctly classify rice types.

Under Time taken we have

Training Time

The total time the model takes to learn from the rice images. It shows how long it takes for the model to understand rice types.

Inference Time Per Image

Time taken to predict the rice type for one image. It indicates how quickly the model can make predictions.

Under Memory we have

Memory Used Before Training

The amount of memory before training begins. It shows the initial computer resource usage.

Memory Used During Training

The amount of memory used during training. It shows how much memory is needed to train the model.

Experimental methodology abstract

Way forward with the experimentation

- We will **build a secondary model on top of a primary pre trained model available on MATLAB.**
- **We are comparing 5 different pre-trained models. AlexNet, GoogLeNet, Vgg 19, DenseNet 201, NasNet Large.** We will be training **two different models of GoogLeNet, one pretrained on ImageNet dataset and one on places 365 dataset.** This will also help us gain insights on the difference between the datasets. The remaining 4 models were trained on the ImageNet dataset.
- Each model will be **tested with different learning parameters** (optimizer, initial learning rate, learn rate schedule, and the minibatch size) **to identify the best synergy of parameters**
- Once selected, we will **coalesce the parameters and look at their convergence more closely to gain further insights** which will allow us to make hypothesis for when we train the model on the entire 75,000 rice image dataset

Key note

We will be using 100 images per class (total 500) for experimentation to conserve time.

The testing / validation split will be 70/30 for most cases (except NasNet Large)

The other parameters will be changed to default to maintain a level playing field while testing out new parameters

The epochs and iterations will be adjusted automatically by the model based on the minibatch size and the size of the dataset

Information on pre trained models and datasets

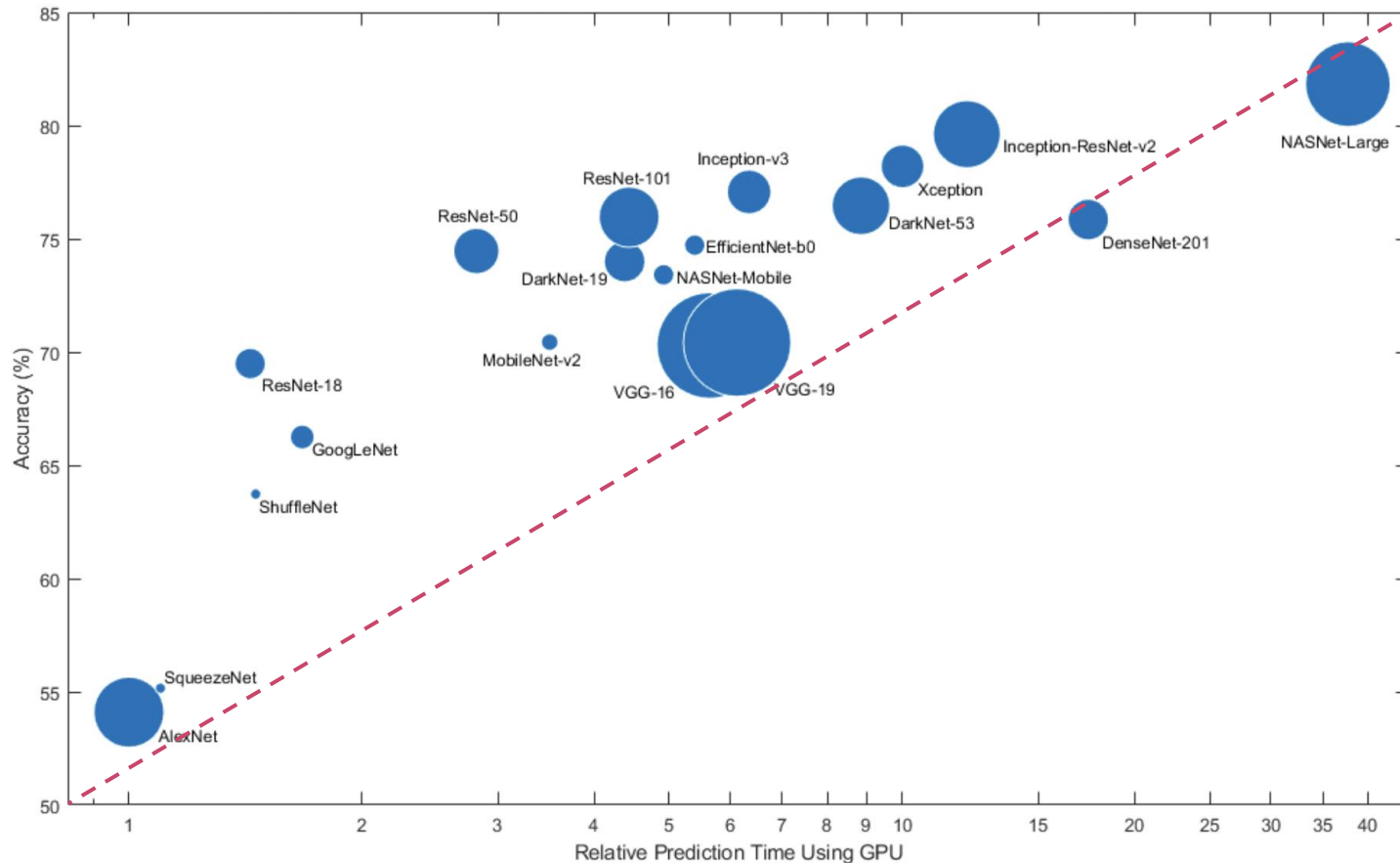
Key note

We have decided to go with the **5 pre trained models** which fit roughly close on the diagonal

This will ensure that we gain an **overview of the pre trained models** we have available on MATLAB

Except **GoogLeNet** which has a **version which is pre trained on places 365 dataset, the remaining models are trained on the ImageNet dataset.**

ImageNet - Large dataset with 14 million images across 1,000 categories
Places 365 - Contains 1.8 million images of 365 scene categories



AlexNet

	Performance metrics - AlexNet									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	99.33%	99.73%	99.35%	99.33%	99.33%	88.92	0.0062	18445.73	1850.75	10 epochs / 14
sgdm	98.00%	99.20%	98.10%	98.00%	97.97%	149.4	0.0036	19962.71	573.73	10 epochs / 14
rmsprop	98.67%	99.47%	98.71%	98.67%	98.67%	85.8	0.0051	18276.02	1739.81	10 epochs / 14
InitialLearnRate										
0.001	20%	68%	4%	20%	6.67%	104.4	0.0039	18901.9	1770.34	10 epochs / 14
0.0001	98.00%	99.20%	98.10%	98.00%	98.00%	153.66	0.0056	20498	1331.34	10 epochs / 14
0.00001	99.33%	99.73%	99.35%	99.33%	99.33%	153.6	0.0062	18445.73	1850.75	10 epochs / 14
0.000001	80%	92%	78.71%	80%	78.93%	178.2	0.0052	19222.29	78.25	10 epochs / 14
LearnRateSchedule										
none	99.33%	99.73%	99.35%	99.33%	99.33%	89.88	0.0062	18445.73	1850.75	10 epochs / 14
piecewise	99.33%	99.73%	99.35%	99.33%	99.33%	28.92	0.0016	147822.33	2044.22	10 epochs / 14
Minibatchsize										
12	100.00%	100.00%	100.00%	100.00%	100.00%	581.4	0.0035	18515.19	-376.48	10 epochs / 29
25	99.33%	99.73%	99.35%	99.33%	99.33%	88.92	0.0062	18445.73	1850.75	10 epochs / 14
50	99.33%	99.73%	99.35%	99.33%	99.33%	32.22	0.0021	17447	-7.78	10 epochs / 7

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 70/30
 Validation frequency - 1

Observations

The fastest pre-trained
 network so far
 Performed the best with low
 MiniBatch size

GoogLeNet - ImageNet

	Performance metrics - GoogLeNet - ImageNet									
	Performance					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
sgdm	94.67%	97.87%	95.32%	94.67%	94.69%	112.62	0.006	20791.61	1558.46	10 epochs / 14
rmsprop	98%	99.20%	98.18%	98%	98.02%	222.3	0.0089	21489.73	-836.63	10 epochs / 14
InitialLearnRate										
0.001	20%	68%	4%	20%	6.67%	180.78	0.0082	20385	440.84	10 epochs / 14
0.0001	98%	99.20%	98.06%	98%	98%	196.98	0.0076	20278.69	912.03	10 epochs / 14
0.00001	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
0.000001	53.33%	81.33%	53.03%	53.33%	52.44%	147.27	0.004	18123.88	-449.53	10 epochs / 14
LearnRateSchedule										
none	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
piecewise	98.00%	98.20%	98.10%	98.00%	98.00%	127.38	0.0033	17138.59	1229.73	10 epochs / 14
Minibatchsize										
12	99.33%	99.73%	99.35%	99.33%	99.33%	232.2	0.0063	17712.43	3255.6	10 epochs / 29
25	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
50	96.67%	98.67%	97%	96.67%	96.65%	2833.2	0.0075	20443.52	147.53	10 epochs / 7

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 70/30
 Validation frequency - 1

Observations

Performed better than its sister
 model which is trained on
 places 365
 Had multiple best performers
 which were based on the
 default parameters

GoogLeNet - places 365

	Performance metrics - GoogLeNet - places 365									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	99.30%	99.73%	99.35%	99.33%	99.34%	51.53	0.0053	13492.49	1718.41	10 epochs / 14
sgdm	88.67%	95.47%	90.81%	88.67%	89.73%	43.49	0.0047	16962.11	1771.8	10 epochs / 14
rmsprop	100.00%	100.00%	100%	100.00%	100.00%	45.75	0.0067	11817.86	3712.27	10 epochs / 14
InitialLearnRate										
0.001	20%	68%	4%	20%	6.67%	41.56	0.0046	14043.29	1914.5	10 epochs / 14
0.0001	90.00%	96.00%	91.93	90.00%	90.95%	64.12	0.007	15133.31	949.77	10 epochs / 14
0.00001	99.30%	99.73%	99.35%	99.33%	99.34%	51.53	0.0053	13492.49	1718.41	10 epochs / 14
0.000001	88.67%	99.47%	57.39%	48.67%	52.67%	45.35	0.0049	15091.99	1612.8	10 epochs / 14
LearnRateSchedule										
none	99.30%	99.73%	99.35%	99.33%	99.34%	51.53	0.0053	13492.49	1718.41	10 epochs / 14
piecewise	99.33%	99.73%	99.35%	99.33%	99.34%	47.07	0.0069	13384.22	-130.67	10 epochs / 14
Minibatchsize										
12	100.00%	100.00%	100.00%	100.00%	100.00%	108.3	0.0098	12629.55	2071.88	10 epochs / 29
25	99.30%	99.73%	99.35%	99.33%	99.34%	51.53	0.0053	13492.49	1718.41	10 epochs / 14
50	92.67%	97.07%	92.81%	92.67%	92.74%	32.2	0.0058	13596.81	-562.54	10 epochs / 7

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 70/30
 Validation frequency - 1

Observations

Fell short in giving best performers when compared to sister model
 Parameters of rmsprop optimizer and MiniBatch size of 12 proved to be the best performers

Vgg 19

	Performance metrics - Vgg19									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	98.67%	99.47%	98.75%	98.67%	98.67%	166.55	0.0059	21484.96	983.22	10 epochs/ 14
sgdm	100.00%	100.00%	100.00%	100.00%	100.00%	201.77	0.0082	22262.75	341.61	10 epochs/ 14
rmsprop	98.67%	99.47%	98.71	98.67%	98.67%	174.64	0.0072	21529.6	-2772.60	10 epochs/ 14
InitialLearnRate										
0.001	59.33%	83.73%	59.01%	59.33%	56.41%	86.59	0.0041	16416.23	513.43	10 epochs/ 14
0.0001	98.67%	99.47%	98.71%	98.67%	98.67%	95.01	0.0047	18111.98	427.76	10 epochs/ 14
0.00001	98.67%	99.47%	98.75%	98.67%	98.67%	166.55	0.0059	21484.96	983.22	10 epochs/ 14
0.000001	96.67	98.67%	96.79%	96.67%	96.69%	85.16	0.0044	16068.23	375.22	10 epochs/ 14
LearnRateSchedule										
none	98.67%	99.47%	98.75%	98.67%	98.67%	166.55	0.0059	21484.96	983.22	10 epochs/ 14
piecewise	100.00%	100.00%	100.00%	100.00%	100.00%	133.85	0.006	18151.88	-20.79	10 epochs/ 14
Minibatchsize										
12	100.00%	100.00%	100.00%	100.00%	100.00%	206.07	0.0067	13410.53	3314.88	10 epochs/ 14
25	98.67%	99.47%	98.75%	98.67%	98.67%	166.55	0.0059	21484.96	983.22	10 epochs/ 14
50	100.00%	100.00%	100.00%	100.00%	100.00%	91.94	0.006	17987.54	-16.02	10 epochs/ 14

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 70/30
 Validation frequency - 1

Observations

The fastest pre-trained
 network so far
 Performed the best with low
 MiniBatch size

DenseNet 201

	Performance metrics - DenseNet 201									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs/ 14
sgdm	98.00%	99.20%	98.02%	98.00%	98.00%	331.64	0.0176	18213.4	-2153.25	10 epochs/ 14
rmsprop	100.00%	100.00%	100	100.00%	100.00%	336.22	0.0178	16075.11	-198.99	10 epochs/ 14
InitialLearnRate										
0.001	97.33%	98.93%	97.54%	97.33%	97.33%	345.26	0.0176	17506.95	-1115.94	10 epochs/ 14
0.0001	99.33%	99.73%	99.35%	99.33%	99.33%	346.22	0.0177	16739.23	-246.01	10 epochs/ 14
0.00001	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs/ 14
0.000001	75.33%	90.13%	83.94%	75.33%	76.01%	344.19	0.018	16567.2	-736.2	10 epochs/ 14
LearnRateSchedule										
none	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs/ 14
piecewise	99.33%	99.73%	99.35%	99.33%	99.33%	337.41	0.0166	16874.68	-2015.63	10 epochs/ 14
Minibatchsize										
12	100.00%	100.00%	100.00%	100.00%	100.00%	510.27	0.0218	16250.13	520.56	10 epochs / 29
25	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs/ 14
50	100.00%	100.00%	100.00%	100.00%	100.00%	473.6	0.0177	15669.42	-386.82	10 epochs / 7

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 70/30
 Validation frequency - 1

Observations

By far the most stable
 performance out of all
 Highest number of best
 performers recorded

NasNet - Large

	Performance metrics - NasNetLarge									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Optimizer										
adam	76.00%	90.40%	76.42%	76.00%	74.60%	441.84	0.0358	15140.79	-1827.54	1 epochs / 10
sgdm	24.00%	69.60%	22.27%	24.00%	19.12%	399.82	0.0342	16389	-2149.23	1 epochs / 10
rmsprop	94.80%	97.92%	94.91	94.80%	94.78%	355.1	0.0294	16423.14	-3875.17	1 epochs / 10
InitialLearnRate										
0.001	89.20%	95.86%	90.09%	89.20%	88.84%	359.57	0.029	12606.3	-1063.58	1 epochs / 10
0.0001	99.60%	99.84%	99.61	99.60%	99.60%	354.06	0.0286	13319.25	-1528.49	1 epochs / 10
0.00001	76.00%	90.40%	76.42%	76.00%	74.60%	441.84	0.0358	15140.79	-1827.54	1 epochs / 10
0.000001	22.00%	68.80%	42.37%	22.00%	18.55%	358.03	0.029	13706.61	-178.61	1 epochs / 10
LearnRateSchedule										
none	76.00%	90.40%	76.42%	76.00%	74.60%	441.84	0.0358	15140.79	-1827.54	1 epochs / 10
piecewise	81.20%	92.48%	83.96%	81.20%	81.56%	353.01	0,0285	14343.62	-3315.23	1 epochs / 10
Minibatchsize										
12	86.60%	94.72%	87.74%	86.80%	86.41%	395.53	0.0302	14260.81	1843.12	1 epochs / 19
25	76.00%	90.40%	76.42%	76.00%	74.60%	441.84	0.0358	15140.79	-1827.54	1 epochs / 10
50	66.40%	86.56%	68.27%	66.40%	66.03%	652.88	0.0342	16041.29	-8536.7	1 epochs / 5

Default parameters

Optimizer - adam
 InitialLearnRate - 0.00001
 MiniBatch Size - 25
 Test / validation split - 50/50
 Validation frequency - 3

Observations

The slowest model of all
 Did not achieve any 100%
 performers
 Most complicated architecture
 out of all

Dropout Analysis

	Performance metrics - Dropout Analysis									
	Performance metrics					Time		Memory		Epochs/iteration per epoch
Learning Parameters	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
Alexnet										
0.5	100.00%	100.00%	100.00%	100.00%	100.00%	64.5	0.003	10737.6	3805.38	10 epochs/ 14
0.75	100.00%	100.00%	100.00%	100.00%	100.00%	366.92	0.0034	16964.9	1130.16	10 epochs/ 14
0.9	98.00%	99.20%	98.06	98.00%	98.00%	201.9	0.0038	15790.16	2326.25	10 epochs/ 14
Vgg										
0.5, 0.75	97.33%	98.93%	97.50%	97.33%	97.34%	87.23	0.0046	16372.7	1735.7	10 epochs/ 14
0.75, 0.95	54.67%	81.87%	59.65	54.67%	52.44%	88.4	0.0045	16837.32	-154.82	10 epochs/ 14
0.95, 0.95	20.00%	68.00%	4.00%	20.00%	6.67%	116.4	0.0057	15947.96	-18.81	10 epochs/ 14
DenseNet										
0.5, 0.75	100.00%	100.00%	100.00%	100.00%	100.00%	327.47	0.0196	12812.43	-259.01	10 epochs/ 14
0.75, 0.95	94.70%	97.87%	94.90%	94.67%	94.66%	344.03	0.0167	14021.38	305.44	10 epochs/ 14
0.95, 0.95	84.67%	93.87%	86.89%	84.67%	84.40%	343.56	0.0172	15080.82	-178.56	10 epochs/ 14
Googlenet										
0.5, 0.75	94.00%	97.60%	94.51%	94.00%	93.97%	50.09	0.0032	15437.18	1060.72	10 epochs/ 14
0.75, 0.95	68.67%	87.47%	81.57%	69.67%	68.47%	44.9	0.0033	15087.35	1161.48	10 epochs/ 14
0.95, 0.95	28.67%	71.47%	42.59%	28.67	23.72%	41.54	0.0047	15217.22	98.01	10 epochs/ 14

Default parameters

The parameters were based upon the best parameters we have recorded in each pre trained network model so far during the testing phase

Observations

Except AlexNet's performance, the accuracy of all remaining models exponentially plummeted

Comparison between best models and AlexNet trained with 75,000 images

	Performance metrics - Best recorded data of all pre trained models									
Model Name	Performance metrics					Time		Memory		Epochs/iteration per epoch
	Overall validation accuracy	Macro accuracy	Macro Precision	Macro Recall	Macro F 1 Score	Training time (sec)	Inference time per image (sec)	Initial toatal memory used (MB)	Total memory used for training (MB)	
AlexNet	100.00%	100.00%	100.00%	100.00%	100.00%	581.4	0.0035	18515.19	-376.48	10 epochs / 29
GoogLeNet - ImageNet	100.00%	100.00%	100.00%	100.00%	100.00%	66.78	0.003	12883.03	1792.89	10 epochs / 14
GoogLeNet - places 365	100.00%	100.00%	100.00%	100.00%	100.00%	45.75	0.0067	11817.86	3712.27	10 epochs / 14
Vgg 19	100.00%	100.00%	100.00%	100.00%	100.00%	201.77	0.0082	22262.75	341.61	10 epochs/ 14
DenseNet 201	100.00%	100.00%	100.00%	100.00%	100.00%	344.06	0.0211	18635.03	-2063.13	10 epochs/ 14

Default parameters

The parameters were based upon the best parameters we have recorded in each pre trained network model so far during the testing phase

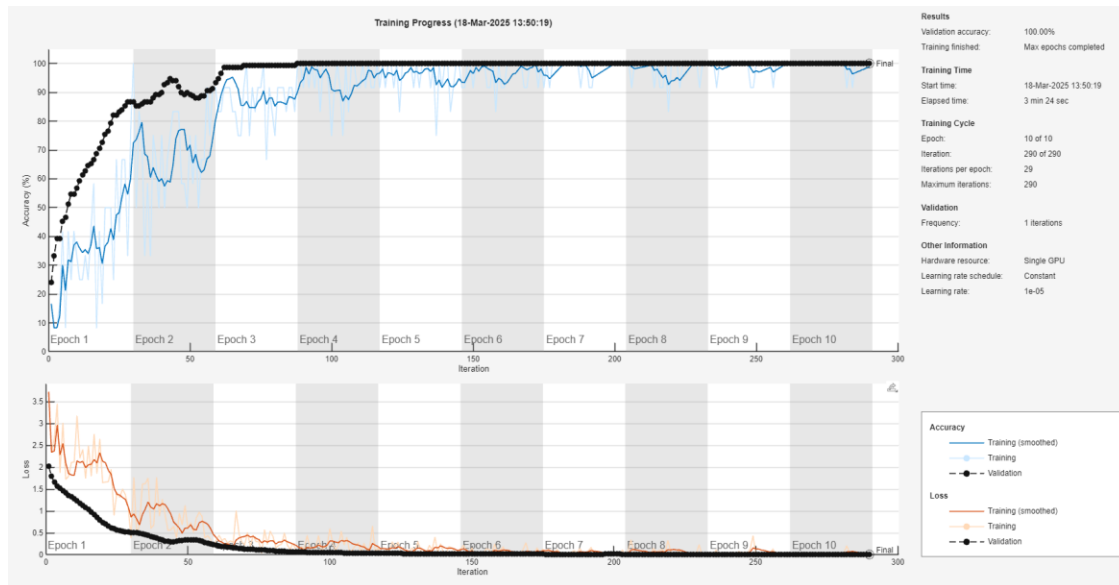
	Performance metrics - AlexNet trained with 75,000 images						
Learning Parameters	Accuracy	Precision	Recall	F 1 Score	Training time (min)	Inference time per image (sec)	Iterations per epoch/epoch
Optimizer							
adam	0.9949	0.995	0.9949	0.9949	14.26	0.0007	52
sgdm	0.9838	0.984	0.9838	0.9838	14.1	0.001	52
rmsprop	0.9863	0.9865	0.9863	0.9863	15.97	0.0013	52
InitialLearnRate							
0.1	0.2	NaN	0.2	NaN	10.7	0.0011	52
0.01	0.2	NaN	0.2	NaN	14.89	0.0009	52
0.001	0.2016	Nan	0.2016	NaN	14.61	0.0012	52
0.0001	0.9952	0.9953	0.9952	0.9952	11.9	0.0014	52
0.00001	0.9888	0.989	0.9888	0.9888	13.71	0.0009	52
0.000001	0.861	0.8621	0.861	0.8597	9.36	0.0006	52
LearnRateSchedule							
none	0.996	0.996	0.996	0.996	13.81	0.001	52
piecewise	0.9959	0.9959	0.9959	0.9959	9.2	0.0007	52
Training validation split							
50/50	0.9952	0.9952	0.9952	0.9952	13.76	0.0006	52
60/40	0.9963	0.9963	0.9963	0.9963	14.18	0.0012	52
70/30	0.9961	0.9962	0.9961	0.9961	9.961	0.0006	52
80/20	0.9952	0.9953	0.9952	0.9952	15.58	0.001	52

Observations

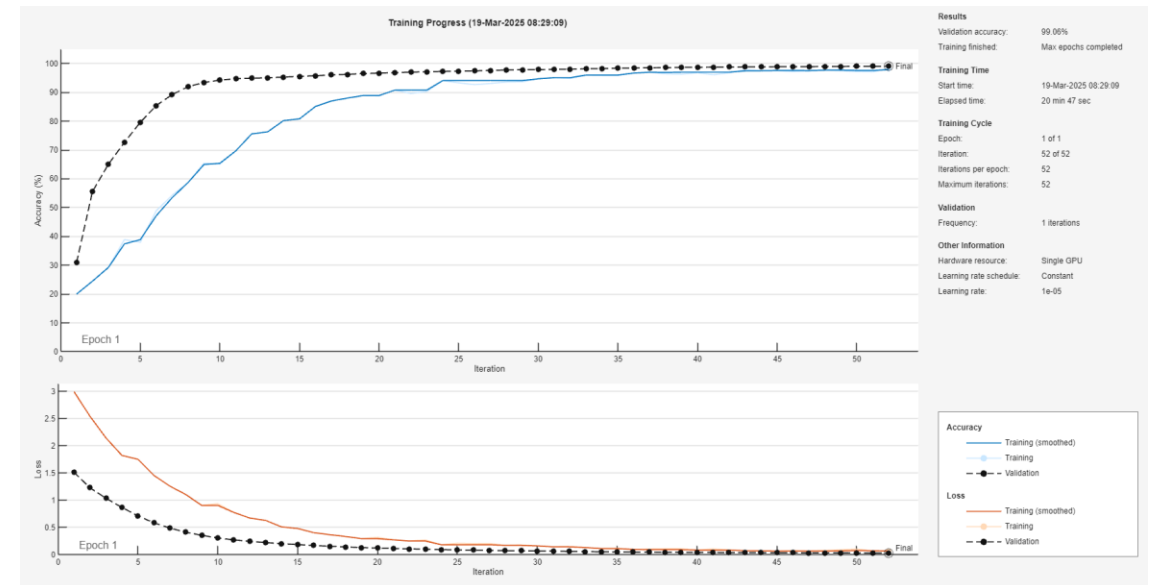
All 100% performers
You can see a strong co relation between the parameters during training sessions using different dataset size (500 and 75,000) for AlexNet

Convergence graphs analysis

AlexNet

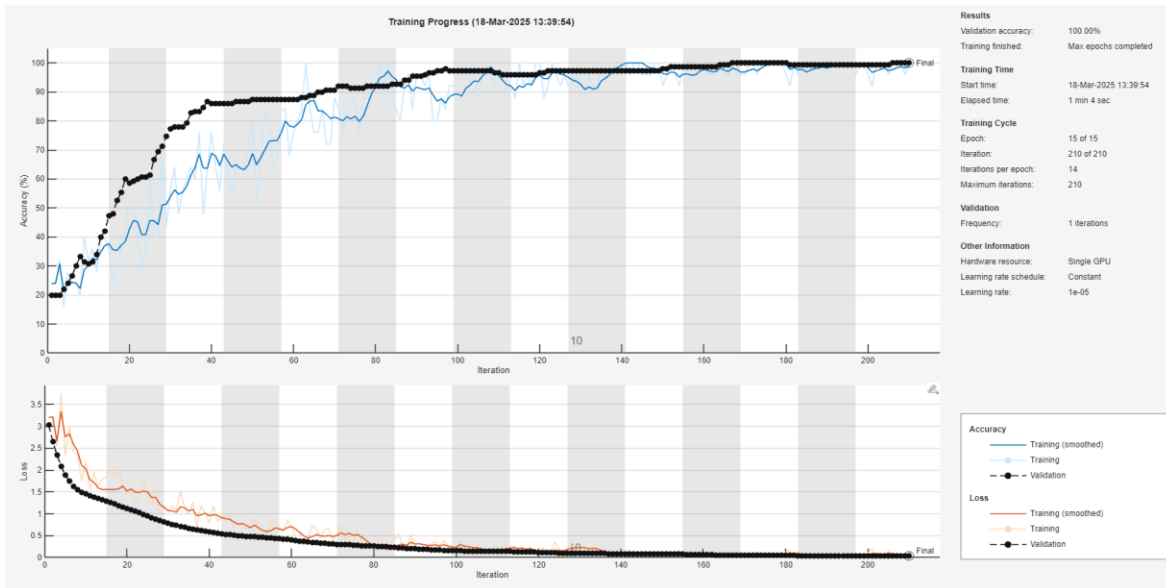


AlexNet with 75,000 images

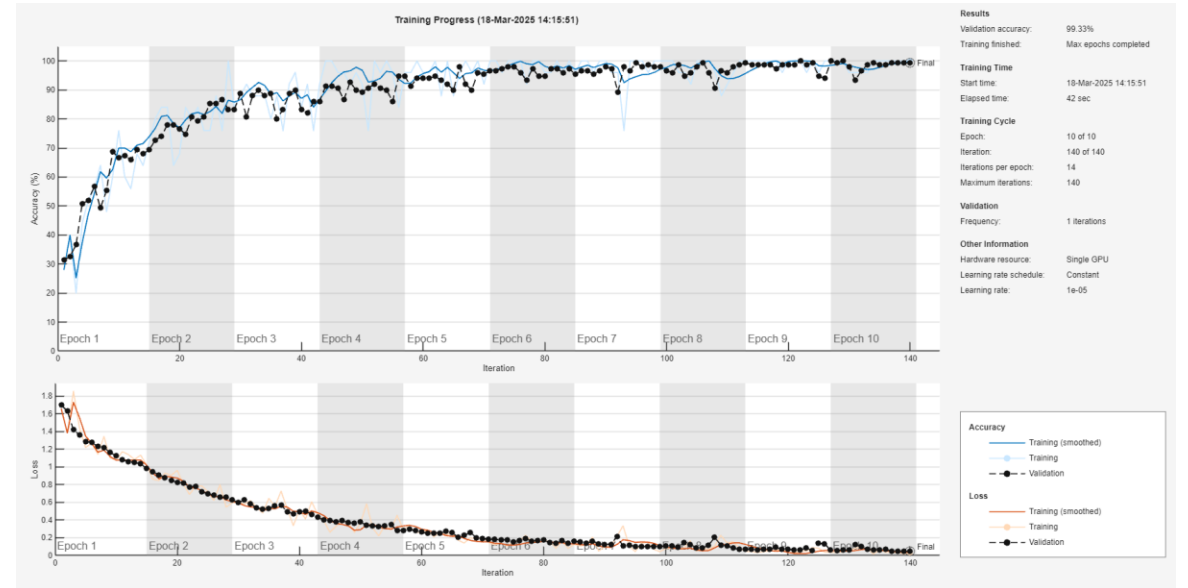


Convergence graphs analysis

GoogLeNet - ImageNet

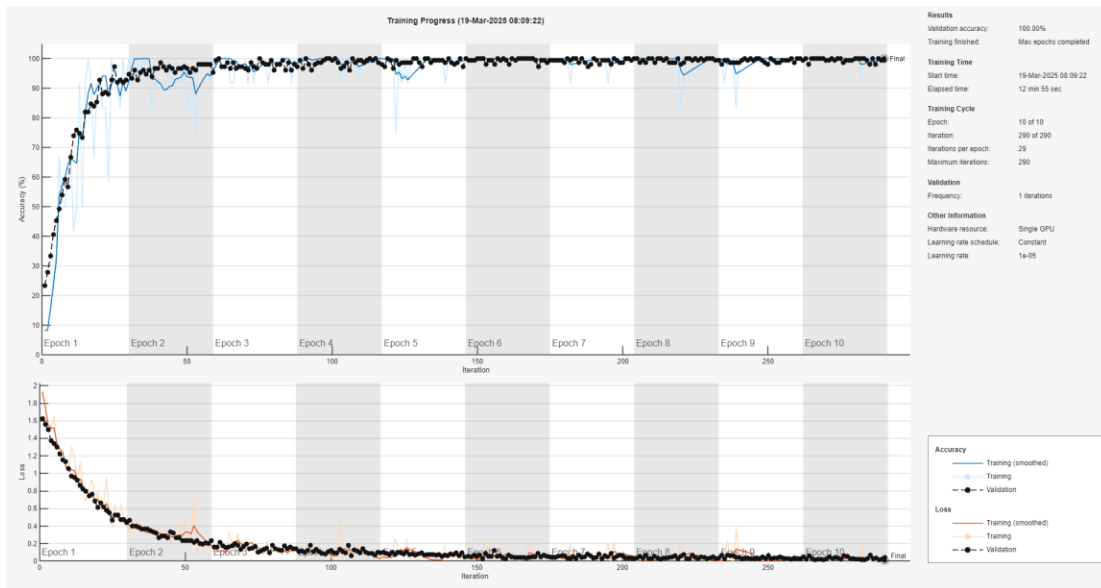


GoogLeNet - places 365

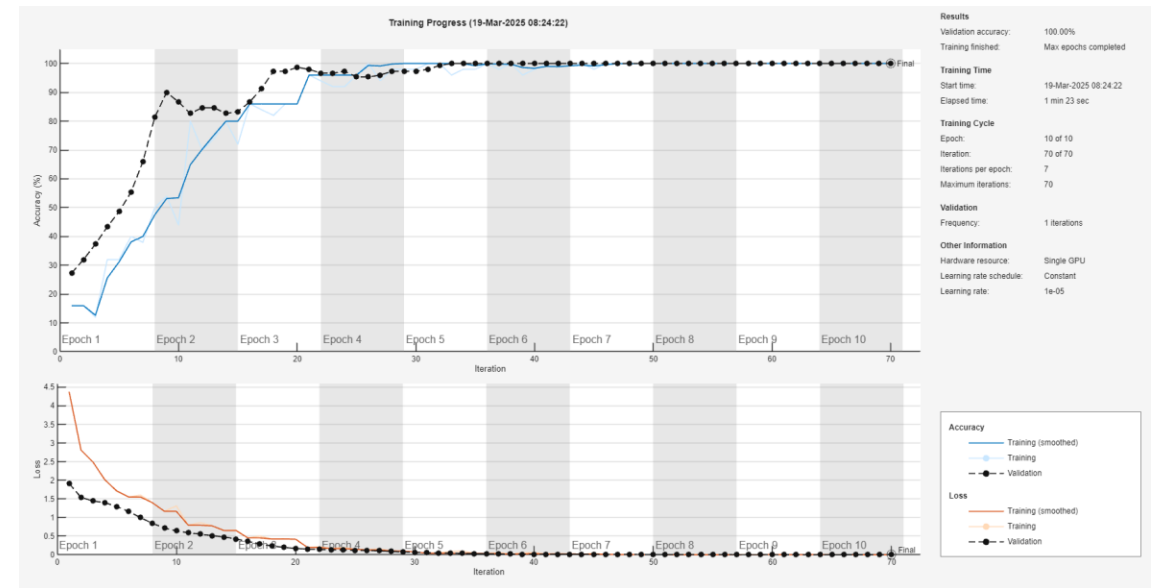


Convergence graphs analysis

DenseNet 201

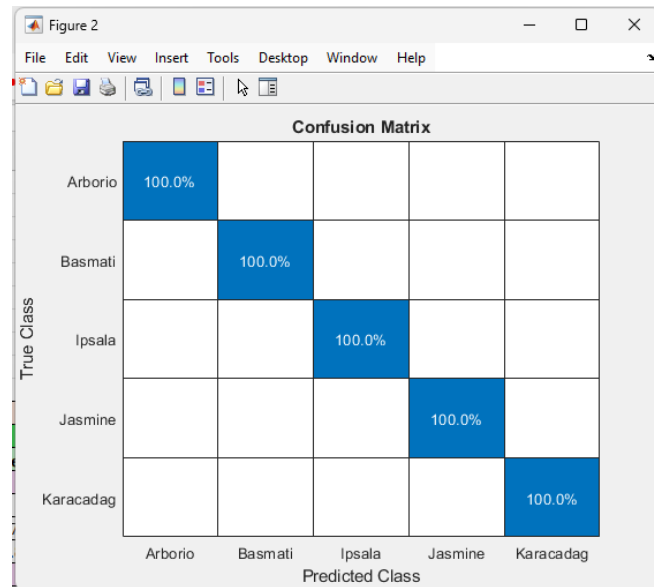


Vgg 19

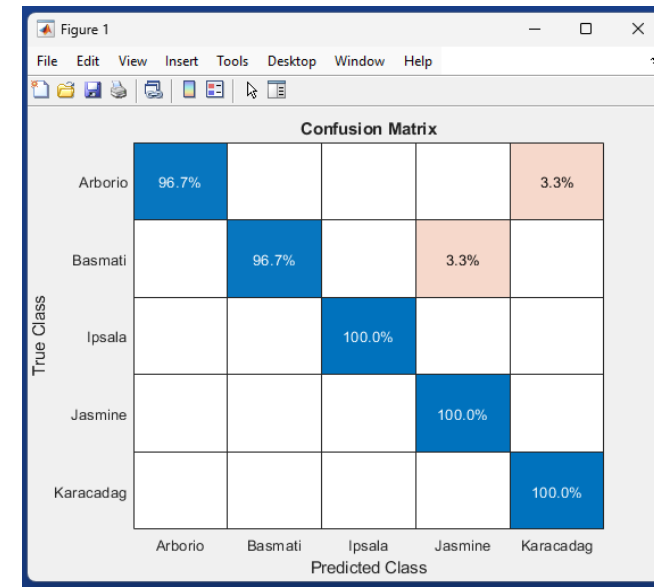


Confusion matrices

100% accuracy



99.06% accuracy



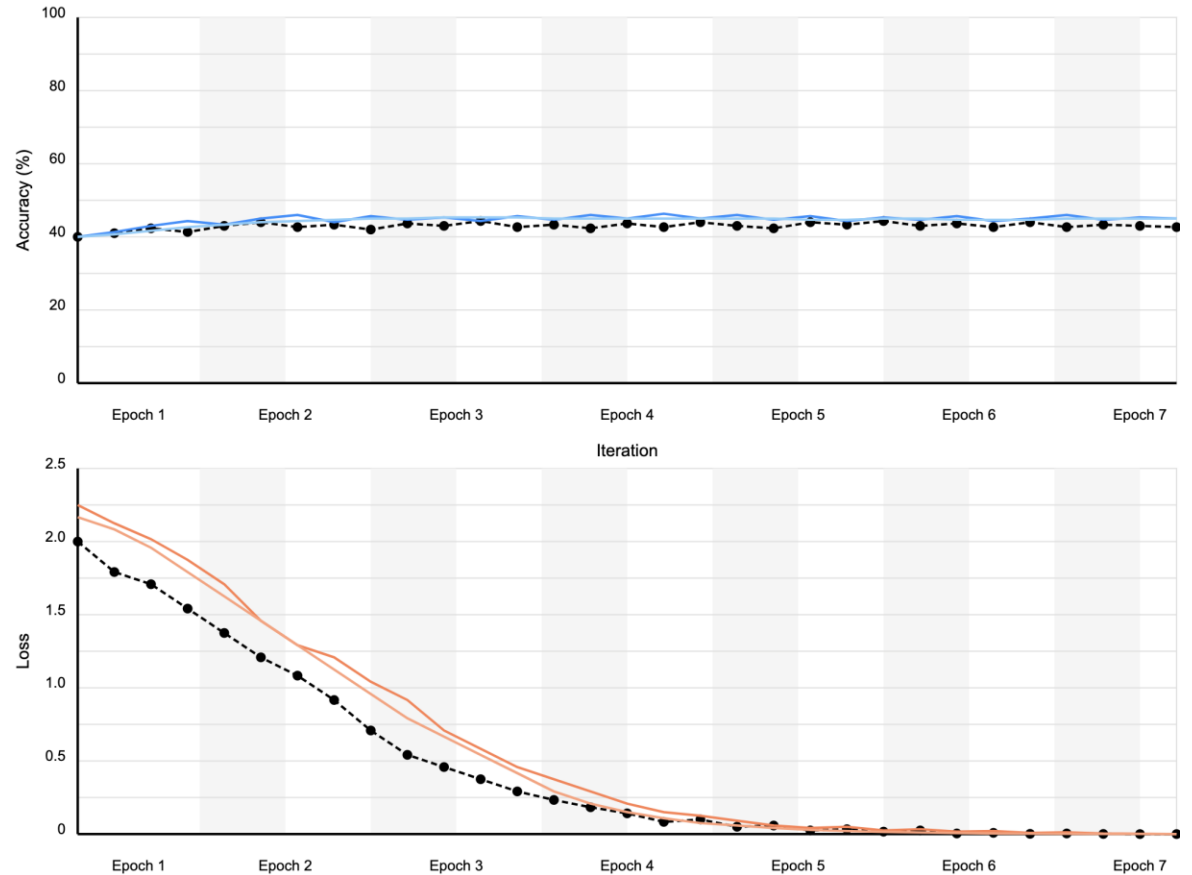
Results

- For larger datasets (like in our case), **training the model beyond a single epoch becomes redundant** since successful convergence is achieved well in advance.
- **MiniBatch size is directly proportional to memory required** (very high MiniBatch size makes MATLAB "hang" and flashes the error, "GPU low on memory").
- **100% accuracy was achieved with all models tested except NasNet Large** due to time and resource constraint
- The **accuracy of the models has a deviation of 0.5% - 1.2% on an average** with each training session which makes our experiment biased in nature
- Except DenseNet 201, all remaining models have smooth training graphs. This indicates that the **model is still making an effort to make itself more robust by trying to identify newer patterns** even in the latter stages of the training.
- During the dataset comparison, we know that the architecture of the GoogLeNet models were same. This aims that the issue lies with the pre training done signifying that **ImageNet is a more effective dataset than places 365 in our problem scenario.**
- Overall good convergence was achieved across all models with **multiple 100% performers.**

Results

Performance metric loophole

The loss function graph is a poor performance metric at times. In a nutshell, it indicates how well the model is learning. In some cases, the model might have very low accuracy, but it might display a 0% loss in the graph (right) indicating that **model is learning wrong patterns over time and becoming confident with its erroneous predictions.** If looked at the graph alone, it might be misleading while assessing the performance of the model.



Conclusions

- Despite having one of the **best resources in terms of GPUs, CPUs, and computational power (hardware)**; the process still proves to be very taxing in **terms of time taken while testing with complex pre trained models and very large datasets.**
- The problem in our case was simple and straightforward in nature and therefore passing it through a heavy pre trained model was **merely nothing but overengineering since 100% was achieved with AlexNet** (the simplest of the models)
- While accuracy and memory can still be tackled with fine tuning and informed resource employment, **time is the most crucial and most difficult aspect out of the three to deal with**
- Performance metrics need to be selected carefully to avoid leading to faulty analysis as seen with loss function graph plotted
- ImageNet proves to be a better starting point than places 365 in our case since **ImageNet specializes in object recognition while places 365 is focused more on scene recognition** making ImageNet a better fit because we are dealing with an object recognition (rice images) task. This was also confirmed by the results we obtained in the dataset comparison done on GoogLeNet model.

Future work

- Time is the crux while working with CNNs. Further development is required in the architecture of the pre trained models which specialize in speed, resource intensiveness, accuracy, and overall computational efficiency. **A balance of the trio will be desirable but is challenging because there are tradeoffs while trying to increase any one of the aspect** (memory, time, and accuracy).
- **A model can be created from scratch which specializes in identifying agricultural objects** to further streamline the pre trained model which will lead to a more confident accuracy percentage since we have a high deviation rate right now because our pre trained is very diverse in nature regarding object recognition.
- If we ever complicate our existing problem, we can investigate complicated models like NasNet Large etc. However, right now **experimenting with complicated models is a waste of resources since we can achieve 100% genralization which require comparatively lower computational power and time.**
- Given enough time, we could run the other models with 75,000 images and see the result.